

LLM-Enhance: Fine-Tuning Large Language Models for Enhanced Detection of Common Weakness Enumerations

Shrey Modi

Dept. of Computer Eng. & Computer Sci.
California State University, Long Beach
Long Beach, CA, United States
shreydharmendra.modi01@student.csulb.edu

Rahul Vishwakarma

Dept. of Computer Eng. & Computer Sci.
California State University, Long Beach
Long Beach, CA, United States
rahuldeo.vishwakarma01@student.csulb.edu

Amin Rezaei

Dept. of Computer Eng. & Computer Sci.
California State University, Long Beach
Long Beach, CA, United States
amin.rezaei@csulb.edu

Abstract—Detecting Common Weakness Enumerations (CWEs) in computational systems remains a challenge for hardware and software developers. In this paper, we scrutinize the efficacy of machine learning algorithms in addressing the above issue, employing both real and synthetic data generated via data augmentation techniques, generative adversarial networks, and rule-based synthesis. Furthermore, we propose fine-tuning Large Language Models (LLMs) to generate two high-quality CWE datasets of 50,000 entries each. We then deploy traditional machine learning algorithms such as support vector classifier, random forest, and naive Bayes, as well as advanced methods such as convolutional neural networks, graph neural networks, and transformer-based architectures, on the curated dataset to perform the task of CWE detection. Results show that fine-tuned LLMs and transformer architectures outperform others, demonstrating their effectiveness in identifying vulnerabilities across hardware-software boundaries.

Index Terms—Common Weakness Enumeration; Machine Learning; Large Language Models; Vulnerability Detection

I. INTRODUCTION

Common Weakness Enumerations (CWEs) are critical vulnerabilities in hardware and software systems [1]. This paper presents a Machine Learning (ML) and Large Language Model (LLM)-based framework for CWE detection using both real and synthetic data. We construct two large CWE datasets enhanced via data augmentation and LLM fine-tuning, and evaluate different ML models on these datasets.

Recent studies emphasize the advantages of ML over traditional static analysis for early vulnerability detection, including applications in Internet of Things (IoT) [2] and log-based anomaly detection [3]. LLMs have shown promise in extracting vulnerability information from unstructured sources [4], zero-shot vulnerability repair [5], code vulnerability detection [6], policy-based protection [7], and hardware security assertion generation [8]. However, challenges remain, such as prompt sensitivity and incomplete responses. Despite these limitations, LLMs demonstrate strong potential in automating security tasks, including penetration testing [9] and solving offensive security challenges [10].

Fig. 1 illustrates the selected CWEs that lie at the intersection of software and hardware vulnerabilities, providing context for the scope of our dataset and analysis.

II. LLM-ENHANCE

In this section, we propose **LLM-Enhance**, a comprehensive workflow designed for hardware and software vulnerability detection shown in Fig. 2. First, we gather and augment CWE data. The collected data is then fine-tuned with LLMs to create two distinct datasets with thousands of data points. These enhanced datasets then undergo vulnerability detection using both traditional ML techniques and state-of-the-art advanced architectures.

A. Data Generation

In the initial phase of our research, we collect approximately 2,270 data points related to five CWEs including CWE-212, CWE-226, CWE-311, CWE-319 and CWE-459 from the DiverseVul [11] and NIST SARD [12] datasets. We then expand the dataset to 5,000 high-quality entries using advanced synthetic data generation tools such as Gretel [13], CTGAN [14], and PALETTE [15].

Data transformation enhances compatibility across environments. For Llama [16] and Falcon [17], this involves converting function source code and reinterpreting vulnerabilities as CSV instructions. Mistral [18] requires preserving columnar structure in JSONL format, while StarCoder [19] focuses on structuring raw code into syntax trees and dependency graphs.

Pre-trained LLMs based on the transformer architecture [20] are initially trained on massive datasets, enabling them to perform a wide range of language tasks such as translation,

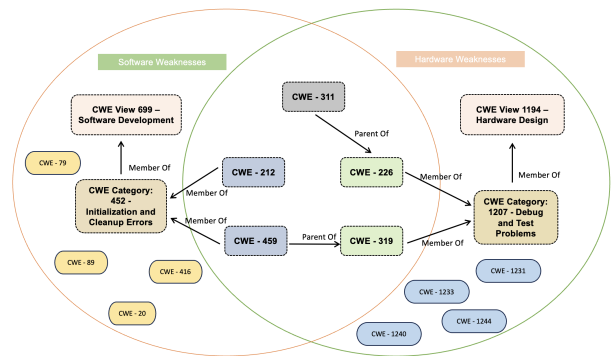


Fig. 1: CWEs at the Intersection of Software and Hardware Weaknesses

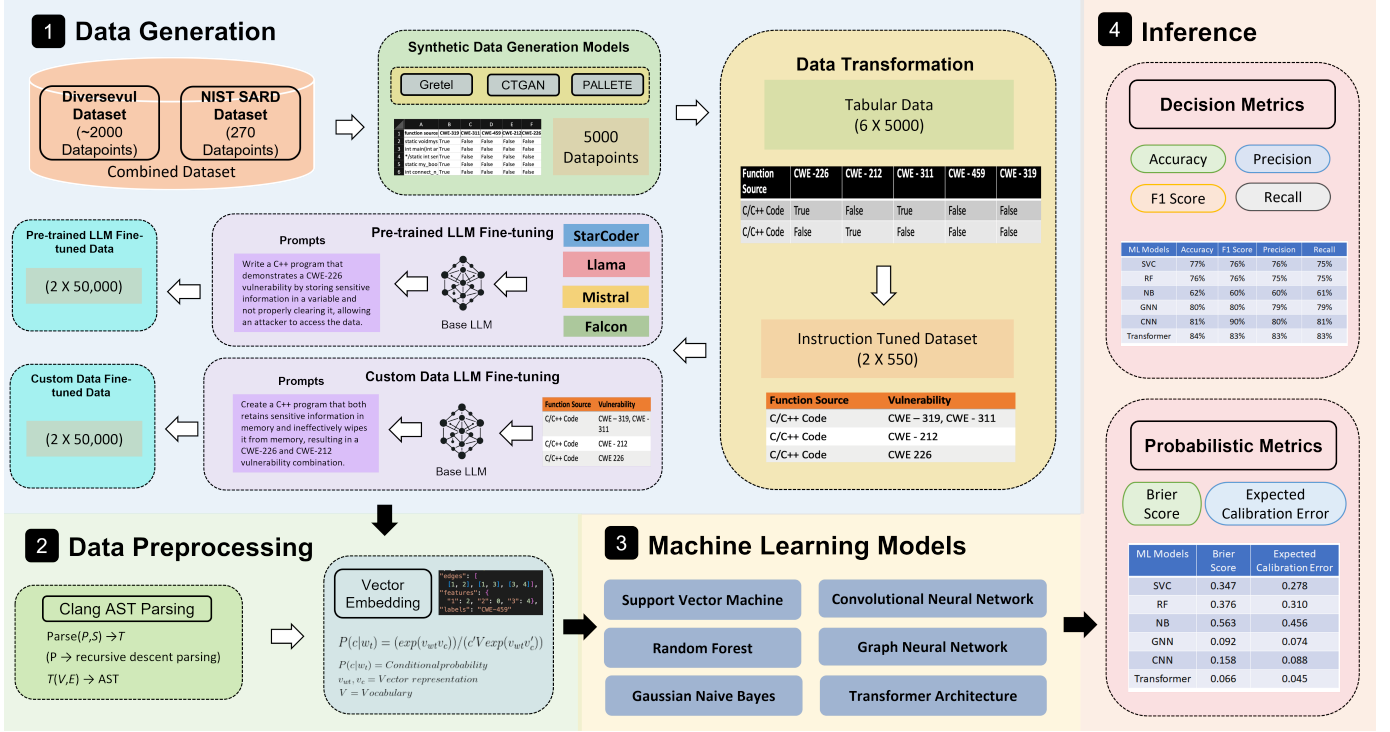


Fig. 2: LLM-Enhance Workflow

summarization, and question answering. Despite their broad capabilities, these models often require fine-tuning for domain-specific applications. The goal of fine-tuning is to minimize cross-entropy loss [21], defined as:

$$\text{Loss} = - \sum_{i=1}^N y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \quad (1)$$

Where N is the number of samples, y_i is the actual label, and $\hat{y}_i$ is the predicted probability. Parameter updates are implemented via gradient descent, optimized by the Adam optimizer [22]:

$$\theta_{t+1} = \theta_t - \eta \cdot \text{Adam}(\nabla_{\theta} \text{Loss}) \quad (2)$$

Here, η represents the learning rate. Through this fine-tuning process, the LLM not only adapts to the specifics of hardware and software vulnerabilities but also enhances its prediction accuracy.

After the data fine-tuning is done, we generate a dataset of about 50,000 data points for each dataset (i.e., the custom LLM dataset and the pre-trained LLM dataset).

B. Data Preprocessing

After the datasets have been generated from the fine-tuned LLM models, we transform C++ code snippets from the function source column into Abstract Syntax Trees (ASTs) using Clang, a specialized compiler front end for the C language family. These ASTs provide a structured and comprehensive analysis of the code, making it easier to identify and interpret various programming constructs. After generating ASTs, the

next step involves converting them into JSON format. JSON's flexibility, manageability, and lightweight structure simplify our preprocessing workflow and enhance the depth and relevance of subsequent analyses.

C. Machine Learning Algorithms

We evaluate six machine learning algorithms on our curated datasets to compare their effectiveness in vulnerability detection.

Support Vector Classifier (SVC) [23] separates classes by constructing a hyperplane that maximizes the margin. It uses kernel functions and regularization to handle complex classification tasks. Random Forest (RF) [24] builds an ensemble of decision trees trained on random subsets of data. It improves robustness and accuracy by aggregating predictions from multiple trees. Gaussian Naive Bayes (GNB) [25] is suitable for high-dimensional data. It applies Bayes' theorem with Gaussian assumptions to estimate class probabilities based on feature distributions. Convolutional Neural Networks (CNNs) [26] are effective at detecting patterns in structured data such as source code. Our CNN uses multiple convolutional and pooling layers followed by dense layers and dropout for regularization. Graph Neural Networks (GNNs) [27] model relationships in graph-structured data. They use message passing between nodes to learn complex dependencies, making them ideal for vulnerability detection in software and hardware systems. Transformers like BERT [28] use self-attention mechanisms to capture bidirectional context in text. Pre-trained on large corpora, they are fine-tuned for specific tasks such as classification and question answering.

TABLE I: Decision Metrics for Pre Trained LLM Dataset

ML Models	Accuracy	F1 Score	Precision	Recall
SVC	46%	43%	43%	44%
RF	44%	43%	43%	42%
GNB	37%	36%	36%	35%
GNN	48%	45%	46%	47%
CNN	47%	45%	46%	46%
Transformer	53%	53%	53%	52%

TABLE II: Decision Metrics for Custom LLM Dataset

ML Models	Accuracy	F1 Score	Precision	Recall
SVC	77%	76%	76%	75%
RF	76%	76%	75%	75%
GNB	62%	60%	60%	61%
GNN	80%	80%	79%	79%
CNN	81%	80%	80%	81%
Transformer	84%	83%	83%	83%

III. EXPERIMENTAL RESULTS

To evaluate the performance of different ML models on the curated pre-trained and custom fine-tuned datasets, we consider both decision and probabilistic metrics. Each dataset is split into 70% training, 15% validation, and 15% testing to ensure balanced evaluation and robust training. Our training minimizes cross-entropy loss with the Adam optimizer, using a learning rate (η) of 0.001 over 30 epochs. The experiments are run on a 12-core Intel Xeon Silver 4310 processor with 16GB of DDR4 RAM and an NVIDIA 64GB A16 GPU.

A. Decision Metrics

Tables I and II show the decision metrics (i.e., accuracy, F1 score, precision, and recall) for pre-trained and custom fine-tuned LLM datasets respectively. The results reflect the evolution in ML models, highlighting the advancements in performance and architecture. In the first comparison, traditional models like SVC and RF showcased moderate performance. GNB, however, demonstrated a noticeably weaker ability to handle the classification task. More advanced models, including GNNs and CNNs, displayed some improvement, but the transformer architecture clearly outperformed them, establishing a new benchmark for model performance. The second comparison highlights substantial improvements across all models, revealing the superiority of the custom LLM fine-tuning. While we observe almost the same trend in the performance of ML models, CNN shows higher performance compared with GNN in this dataset. The transformer architecture, however, took center stage, excelling across all decision metrics and firmly establishing its superiority.

One notable point is that once we switch Llama 2 to Llama 3, we observe 3% increase in accuracy, from 81% to 84%. Llama 3 utilizes a larger and more diverse training dataset, expanding its understanding of language nuances and context, thereby enhancing its generalization capabilities. Additionally, advanced fine-tuning methods help Llama 3 adapt better to specific tasks.

TABLE III: Probabilistic Metrics for Pre-Trained LLM Dataset

ML Models	Brier Score	Expected Calibration Error
SVC	0.456	0.347
RF	0.487	0.391
GNB	0.620	0.510
GNN	0.342	0.287
CNN	0.320	0.236
Transformer	0.290	0.187

TABLE IV: Probabilistic Metrics for Custom LLM Dataset

ML Models	Brier Score	Expected Calibration Error
SVC	0.347	0.278
RF	0.376	0.310
GNB	0.563	0.456
GNN	0.092	0.074
CNN	0.158	0.088
Transformer	0.066	0.045

B. Probabilistic Metrics

The Brier Score (BS) and Expected Calibration Error (ECE) provide realistic perspectives on the performance of classification models. The BS calculates the squared discrepancies between the predicted probabilities and the actual results, offering a numerical representation of probabilistic prediction accuracy. A model with a lower BS suggests that its probability estimates are closely aligned with the actual outcomes, aiding in evaluating not just the binary correctness but also the confidence of the predictions. In addition, ECE evaluates the extent to which the predicted probabilities mirror the actual probabilities of outcomes. It scrutinizes the calibration of a model's predictions, ensuring that the probabilities accurately correspond to the actual event likelihoods. A model with BS and ECE near zero is considered well-calibrated, meaning its predicted probabilities closely match actual outcomes. These metrics matter most when probability quality is more important than accuracy alone. The formula for BS calculation is as follows:

$$BS = \frac{1}{N} \sum_{i=1}^N (f_i - o_i)^2 \quad (3)$$

Where f_i denotes the predicted probability for an event, o_i indicates the actual outcome (1 or 0), and N is the total number of predictions. The formula for ECE calculation is as follows:

$$ECE = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (4)$$

Where n is the total number of samples, B_m is the set of indices of samples in bin m , $\text{acc}(B_m)$ is the accuracy within bin m , and $\text{conf}(B_m)$ is the mean predicted probability in bin m .

Tables III and IV evaluates the probabilistic metrics (i.e., BS and ECE) for pre-trained and custom fine-tuned LLM dataset respectively. Deep learning models outperform traditional methods, with transformer architecture achieving the lowest BS (0.066) and ECE (0.045) in custom fine-tuned LLM dataset. In contrast, traditional models like GNB exhibit the highest BS (0.563) and ECE (0.456) in custom fine-tuned LLM

